MODULE II — BLIND-SPOT WARNING

```
import random
import time

SIDE_RSSI_ALERT_THRESHOLD_DBM = -55


class EgoVehicle:
    def __init__(self, node_id):
        self.node_id = node_id
        self.indicator_status = 0

    def activate_indicator(self):
        self.indicator_status = 1
        print(
            f"[INPUT] Node {self.node_id}: turn indicator button pressed "
            "-> indicator_status = 1"
        )

    def reset_indicator(self):
        self.indicator_status = 0


def check_side_channel(vehicle_present: bool):
    if vehicle_present:
        return random.randint(-50, -40)
    return random.randint(-95, -75)


def run_blind_spot_check(ego: EgoVehicle, vehicle_present: bool):
    if ego.indicator_status != 1:
        print("[LOGIC] Indicator OFF -> side channel not checked, normal operation.")
        return

    print(f"[RF] Node {ego.node_id}: checking side-facing nRF24L01 channel...")
    time.sleep(0.4)
    rssi = check_side_channel(vehicle_present)
    print(
        f"[RX] Side channel RSSI = {rssi} dBm "
        f"(alert threshold = {SIDE_RSSI_ALERT_THRESHOLD_DBM} dBm)"
    )
    time.sleep(0.3)

    if rssi >= SIDE_RSSI_ALERT_THRESHOLD_DBM:
        print("[ALERT] Vehicle detected in blind spot!")
        print(
            f"[OUTPUT] Node {ego.node_id} -> Buzzer ON | LED: RED | "
            'LCD: "BLIND SPOT" (lane change blocked)'
        )
    else:
        print(
            f"[OUTPUT] Node {ego.node_id} -> Buzzer OFF | LED: GREEN | "
            'LCD: "SAFE" (lane change permitted)'
        )


def run_demo():
    print("=" * 60)
    print("Smart V2V Safety Broadcast System — Module II Demo")
    print("Blind-Spot Warning (software simulation)")
    print("=" * 60)

    ego = EgoVehicle("B")

    while True:
        print("\n" + "-" * 60)
        choice = input(
            "Press [Enter] for indicator OFF, type 'i' for a lane change "
            "with a CLEAR blind spot, type 'iv' if a VEHICLE is in the blind "
            "spot, or 'q' to quit: "
        ).strip().lower()

        if choice == "q":
            print("\nDemo ended.")
            break
```

```
        if choice == "i":
            ego.activate_indicator()
            run_blind_spot_check(ego, vehicle_present=False)
        elif choice == "iv":
            ego.activate_indicator()
            run_blind_spot_check(ego, vehicle_present=True)
        else:
            ego.reset_indicator()
            run_blind_spot_check(ego, vehicle_present=False)

        ego.reset_indicator()


if __name__ == "__main__":
    run_demo()
```